(b) `5 6 7 7 1 7 1 2 2 3 5 3 4`

7. Obtain the normalized chaincodes and shape numbers of all the shapes in the previous questions. Check your answers with `shape`.
8. Use your system to generate a T shape, and obtain its Fourier descriptors. How many terms are required to identify the

   (a) Symmetry of the object?
   (b) Size of the object?
   (c) Shape of the object?

9. Repeat the above question but use an X shape. Then again, but with an E shape.
10. Compare the results of the shapes in the previous two questions. How many terms are required to distinguish the

    (a) Symmetries
    (b) Sizes
    (c) Shapes

of the three objects?
11. How does the size of an object affect its Fourier descriptors? Experiment with a 6×4 rectangle, and then with a 12×8 rectangle. Generalize your findings.

---

13 Color Processing

For human beings, color provides one of the most important descriptors of the world around us. The human visual system is particularly attuned to two things: edges and color. We have mentioned that the human visual system is not particularly good at recognizing subtle changes in gray values. In this section we shall investigate color briefly, and then some methods of processing color images

13.1 What Is Color?

Color study consists of

1. The physical properties of light which give rise to color
2. The nature of the human eye and the ways in which it detects color
3. The nature of the human vision center in the brain, and the ways in which messages from the eye are perceived as color

Physical Aspects of Color

As we have seen in Chapter 1, visible light is part of the electromagnetic spectrum. The values for the wavelengths of blue, green, and red were set in 1931 by the CIE (Commission Internationale d'Eclairage), an organization responsible for color standards.

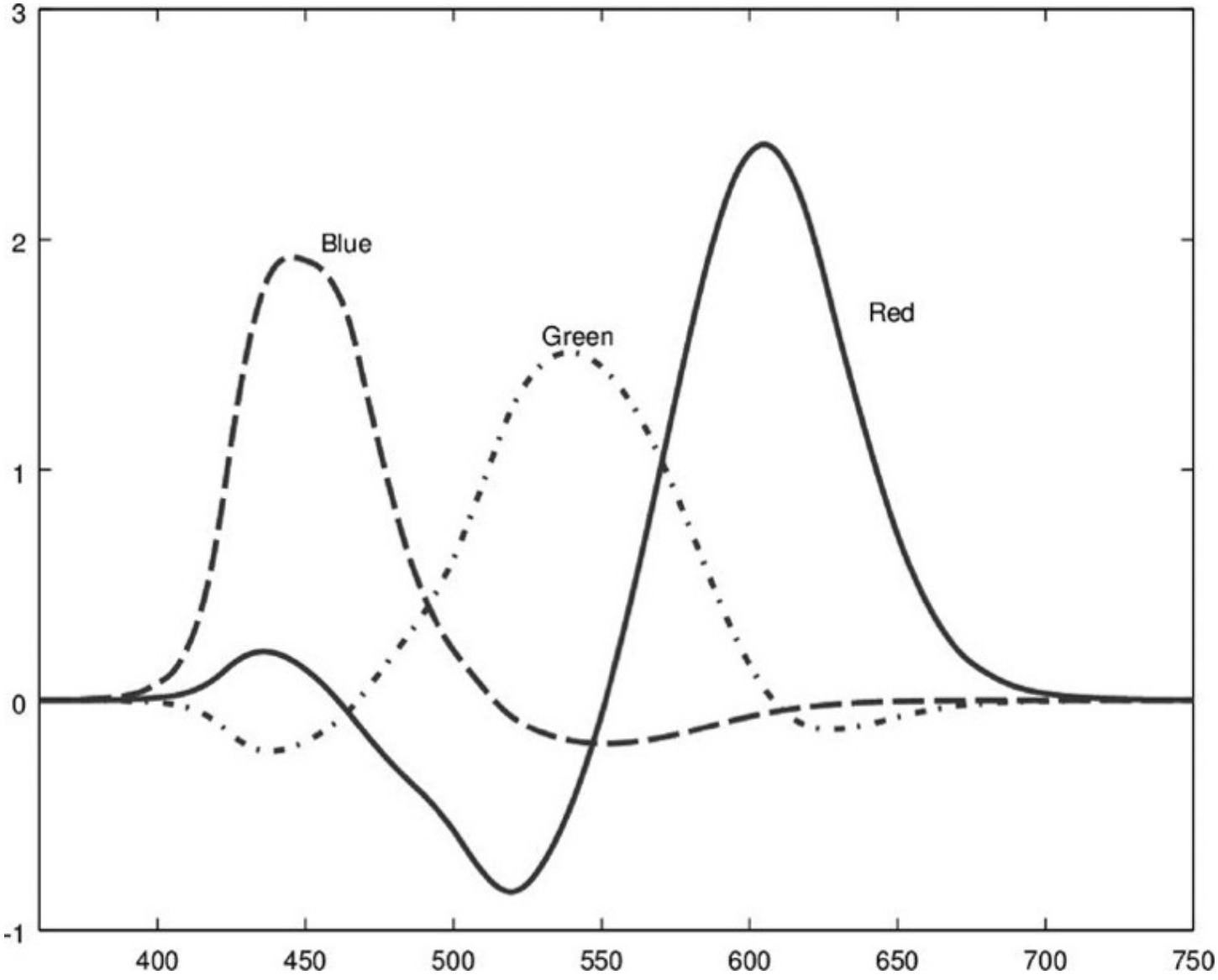
Perceptual Aspects of Color

The human visual system tends to perceive color as being made up of varying amounts of red, green, and blue. That is, human vision is particularly sensitive to these colors; this is a function of the cone cells in the retina of the eye. These values are called the *primary colors*. If we add together any two primary colors, we obtain the *secondary colors*:

$$\begin{aligned} \text{magenta (purple)} &= \text{red} + \text{blue} \\ \text{cyan} &= \text{green} + \text{blue} \\ \text{yellow} &= \text{red} + \text{green} \end{aligned}$$

The amounts of red, green, and blue that make up a given color can be determined by a *color matching experiment*. In such an experiment, people are asked to match a given color (a *color source*) with different amounts of the additive primaries red, green, and blue. Such an experiment was performed in 1931 by the CIE, and the results are shown in Figure 13.1. Note that for some wavelengths, various red, green, or blue values are *negative*. This is a

Figure 13.1: RGB color matching functions (CIE, 1931)



physical impossibility, but it can be interpreted by adding the primary beam to the color source, to maintain a color match.

To remove negative values from color information, the CIE introduced the XYZ color model. The values of *X, Y*, and *Z* can be obtained from the corresponding *R, G*, and *B* values by a linear transformation:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.431 & 0.342 & 0.178 \\ 0.222 & 0.707 & 0.071 \\ 0.020 & 0.130 & 0.939 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

The inverse transformation is easily obtained by inverting the matrix:

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 3.063 & -1.393 & -0.476 \\ -0.969 & 1.876 & 0.042 \\ 0.068 & -0.229 & 1.069 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

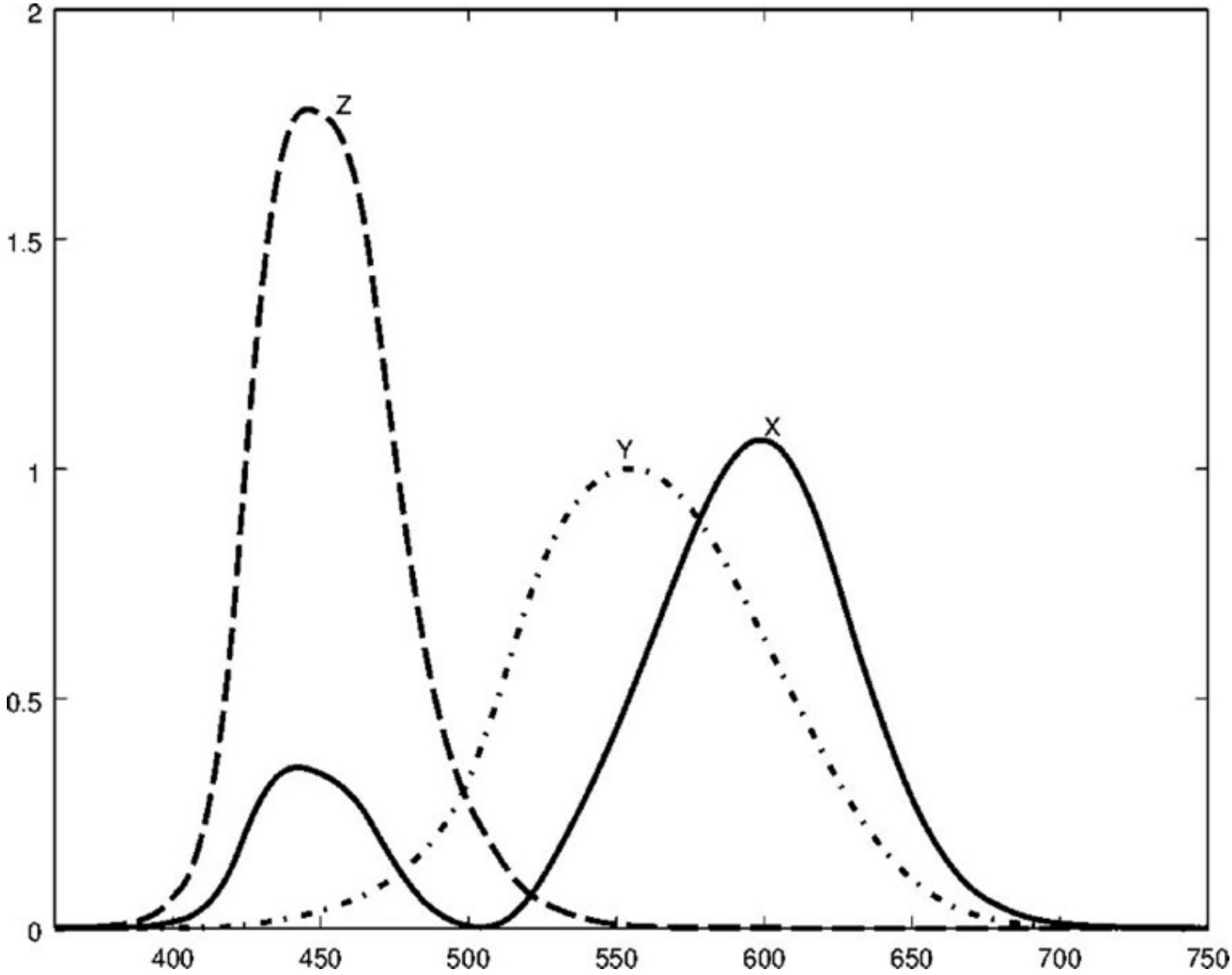
The XYZ color matching functions corresponding to the *R, G, B* curves of Figure 13.1 are shown in Figure 13.2. The matrices given are not fixed; other matrices can be defined according to the definition of the color white. Different definitions of white will lead to different transformation matrices.

The CIE required that the *Y* component corresponded with *luminance*, or perceived brightness of the color. That is why the row corresponding to *Y* in the first matrix (that is, the second row) sums to 1, and also why the *Y* curve in Figure 13.2 is symmetric about the middle of the visible spectrum.

In general, the values of *X, Y*, and *Z* needed to form any particular color are called the *tristimulus values*. Values corresponding to particular colors can be obtained from published tables. In order to discuss color independent of brightness, the tristimulus values can be

normalized by dividing by $X + Y + Z$:

Figure 13.2: XYZ color matching functions (CIE, 1931)



$$x = \frac{X}{X+Y+Z}$$
$$y = \frac{Y}{X+Y+Z}$$
$$z = \frac{Z}{X+Y+Z}$$

and so $x + y + z = 1$. Thus, a color can be specified by x and y alone, called the *chromaticity coordinates*. Given x, y, and Y, we can obtain the tristimulus values X and Z by working through the above equations backward:

$$X = \frac{x}{y}Y$$
$$Z = \frac{1-x-y}{y}Y.$$

We can plot a chromaticity diagram, using the `ciexyz31.csv`[1] file of XYZ values. In MATLAB/Octave:

```
>> nxyz = dlmread('ciexyz31.csv');
>> xyz = wxyz(:,2:4)';
>> xy = xyz'./(sum(xyz)'*[1 1 1]);
>> x = xy(:,1)';
>> y = xy(:,2)';
>> figure,plot([x x(1)],[y y(1)]),xlabel('x'),ylabel('y'),axis square
```

MATLAB/Octave

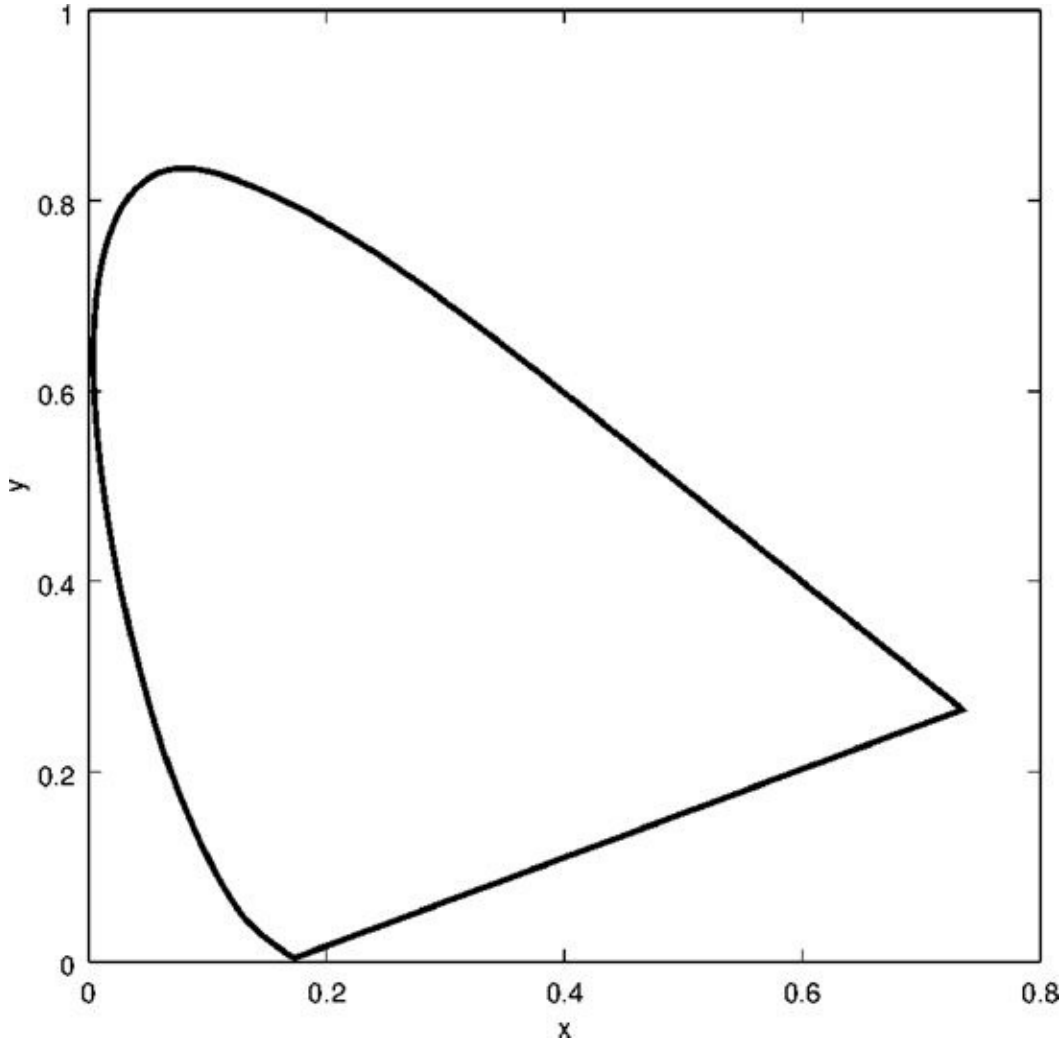
In Python:

```
In :  import matplotlib.pyplot as plt
In :  nxyz = np.loadtxt('ciexyz31_1.csv',delimiter=',')
In :  xyz = nxyz[:,1:]
In :  sums = xyz.sum(axis=1)
In :  newxyz = xyz/sums[:,np.newaxis]
In :  x = newxyz[0,:]; x1 = np.hstack((x,x[0]))
In :  y = newxyz[1,:]; y1 = np.hstack((y,y[0]))
In :  plt.plot(x1,y1)
```

Python

Here the matrix `xyz` consists of the second, third, and fourth columns of the data, and `plot` is a function that draws a polygon with vertices taken from the `x` and `y` vectors. The extra `x(1)` and `y(1)` ensures that the polygon joins up. The result is shown in Figure 13.3. The values of x and y that lie within the horseshoe shape in Figure 13.3 represent values that correspond to physically realizable colors. A good account of the XYZ model and associated color theory can be found in Foley et al. [11].

Figure 13.3: A chromaticity diagram



1

This file can be obtained from the *Color & Vision Research Laboratories* web page **http://www.cvrl.org**.

13.2 Color Models

A *color model* is a method for specifying colors in some standard way. It generally consists of a three-dimensional coordinate system and a subspace of that system in which each color is represented by a single point. We shall investigate three systems.

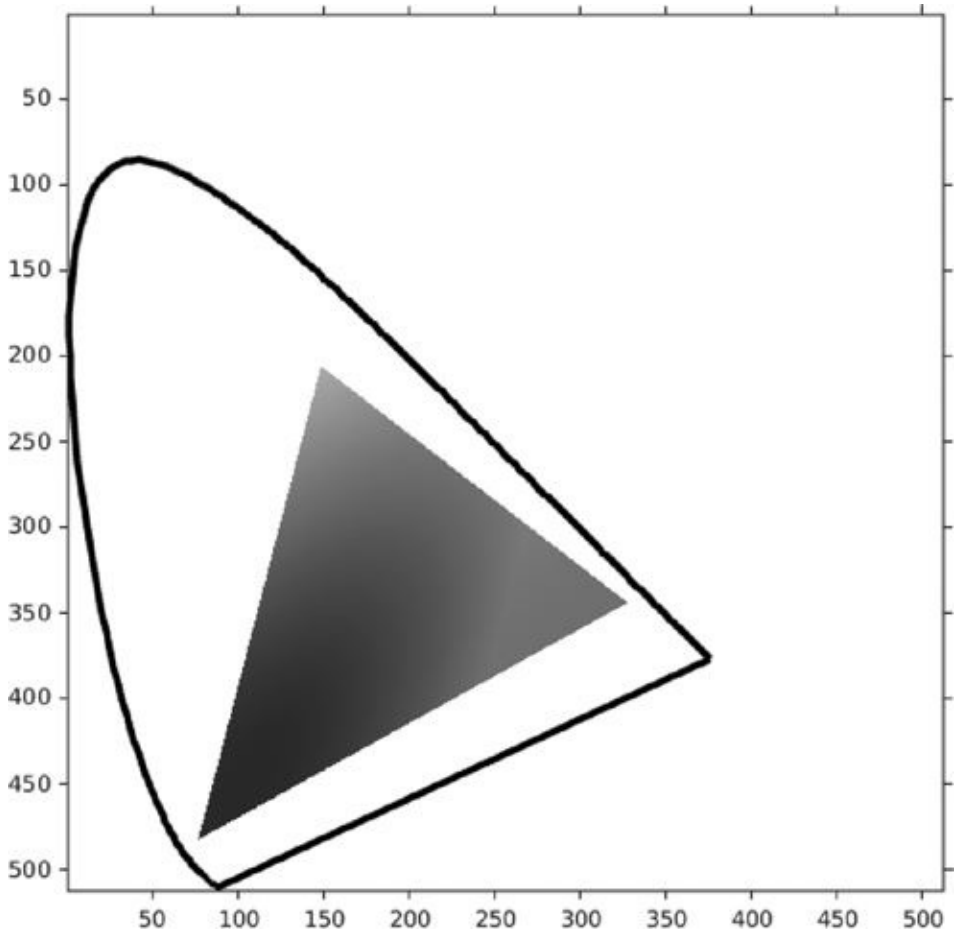
RGB

In this model, each color is represented as three values *R, G*, and *B*, indicating the amounts of red, green, and blue which make up the color. This model is used for displays on computer screens; a monitor has three independent electron "guns" for the red, green, and blue component of each color. We have met this model in Chapter 2.

Note also from Figure 13.1 that some colors require negative values of *R, G* or *B*. These colors are not realizable on a computer monitor or TV set, on which only positive values are possible. The colors corresponding to positive values form the *RGB gamut*; in general a color "gamut" consists of all the colors realizable with a particular color model. We can plot the RGB gamut on a chromaticity diagram, using the

xy coordinates obtained above. To define the gamut, we shall create a 100 × 100 × 3 array, and to each point (i, j) in the array, associate an XYZ triple defined by $(i/100, j/100, 1 - i/100 - j/100)$. We can then compute the corresponding RGB triple, and if any of the RGB values are negative, make the output value white. Programs to display the gamut, which is shown in Figure 13.4, are given at the end of the chapter

Figure 13.4: SEE COLOR INSERT The RGB gamut



HSV

HSV stands for hue, saturation, and value. These terms have the following meanings:

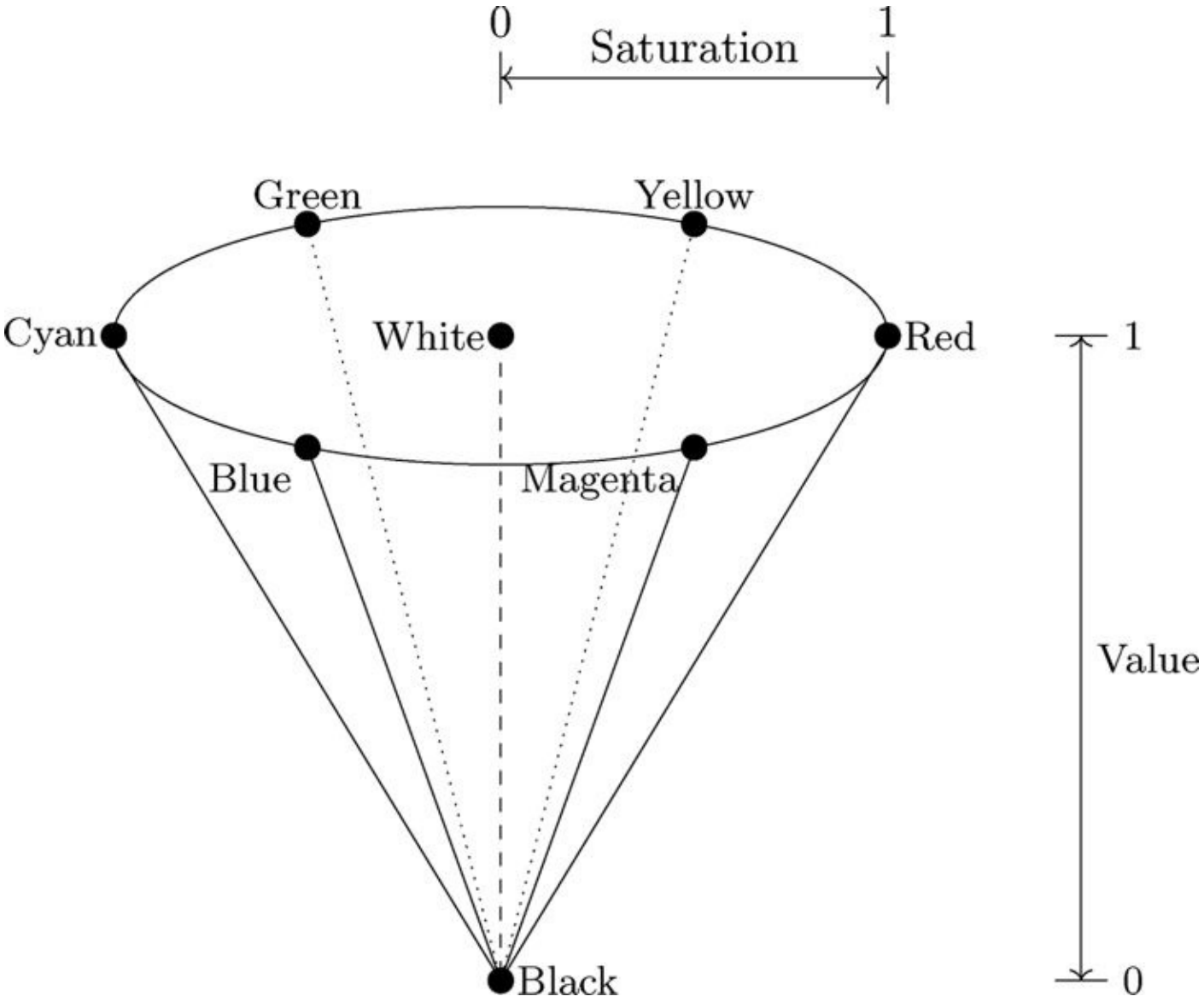
Hue: The "true color" attribute (red, green, blue, orange, yellow, and so on).

Saturation: The amount by which the color has been diluted with white. The more white in the color, the lower the saturation. So a deep red has high saturation, and a light red (a pinkish color) has low saturation.

Value: The degree of brightness: a well-lit color has high intensity; a dark color has low intensity.

This is a more intuitive method of describing colors, and as the intensity is independent of the color information, this is a very useful model for image processing. We can visualize this model as a cone, as shown in Figure 13.5.

Figure 13.5: The color space HSV as a cone



Any point on the surface represents a purely saturated color. The saturation is thus given as the relative distance to the surface from the central axis of the structure. Hue is defined to be the angle measurement from a pre-determined axis, say red.

Conversion between RGB and HSV

Suppose a color is specified by its RGB values. If all three values are equal, then the color will be a grayscale; that is, an intensity of white. Such a color, containing just white, will thus have a saturation of zero. Conversely, if the RGB values are very different, we would expect the resulting color to have a high saturation. In particular, if one or two of the RGB values are zero, the saturation will be one, the highest possible value.

Hue is defined as the fraction around the circle starting from red, which thus has a hue of zero. Reading around the circle in Figure 13.5 produces the following hues:

Color	Hue

Red	0
Yellow	0.1667
Green	0.3333
Cyan	0.5
Blue	0.6667
Magenta	0.8333

Suppose we are given three *R, G, B* values, which we suppose to be between 0 and 1. So if they are between 0 and 255, we first divide each value by 255. We then define:

$$V = \max\{R, G, B\}$$
$$\delta = V - \min\{R, G, B\}$$
$$S = \frac{\delta}{V}$$

To obtain a value for hue, we consider several cases:

1. $\text{if } R = V \text{ then } H = \frac{1}{6}\frac{G - B}{\delta}$
2. $\text{if } G = V \text{ then } H = \frac{1}{6}\left(2 + \frac{B - R}{\delta}\right)$
3. $\text{if } B = V \text{ then } H = \frac{1}{6}\left(4 + \frac{R - G}{\delta}\right)$

If *H* ends up with a negative value, we add 1. In the particular case (*R, G, B*) = (0, 0, 0), for which both *V* = δ = 0, we define (*H, S, V*) = (0,0,0).

For example, suppose (*R, G, B*) = (0.2, 0.4, 0.6) We have

$$V = \max\{0.2, 0.4, 0.6\} = 0.6$$
$$\delta = V - \min\{0.2, 0.4, 0.6\} = 0.6 - 0.2 = 0.4$$
$$S = \frac{0.4}{0.6} = 0.6667$$

Since *B* = *G* we have

$$H = \frac{1}{6}\left(4 + \frac{0.2 - 0.4}{0.4}\right) = 0.5833.$$

Conversion in this direction is implemented by the `rgb2hsv` function, and by the `rgb2hsv` method in the `skimage.color` module in Python. This is of course designed to be used on $m \times n \times 3$ arrays, but let's just experiment with our previous example:

```
>> rgb2hsv([0.2 0.4 0.6])
ans =

    0.5833    0.6667    0.6000
```

MATLAB/Octave

and

```
In :  sk.color.rgb2hsv(np.array([[[0.2,0.4,0.6]]]))
Out: array([[[ 0.58333333,  0.66666667,  0.6       ]]])
```

Python

and these are indeed the *H, S*, and *V* values we have just calculated.

To go the other way, we start by defining:

$$
\begin{aligned}
H' &= \lfloor 6H \rfloor \\
F &= 6H - H' \\
P &= V(1 - S) \\
Q &= V(1 - SF) \\
T &= V(1 - S(1 - F))
\end{aligned}
$$

Since H' is an integer between 0 and 5, we have six cases to consider:

H'	R	G	B
0	V	T	P
1	Q	V	P
2	P	V	T
3	P	Q	V
4	T	P	V
5	V	P	Q

Let's take the HSV values we computed above. We have:

$$
\begin{aligned}
H' &= \lfloor 6(0.5833) \rfloor = 3 \\
F &= 6(0.5833) - 3 = 0.5 \\
P &= 0.6(1 - 0.6667) = 0.2 \\
Q &= 0.6(1 - (0.6667)(0.5)) = 0.4 \\
T &= 0.6(1 - 0.6667(1 - 0.5)) = 0.4
\end{aligned}
$$

Since $H' = 3$ we have

$$(R, G, B) = (P, Q, V) = (0.2, 0.4, 0.6).$$

Conversion from HSV to RGB is implemented by the `hsv2rgb` function in MATLAB and Octave, and in the `skimage.color` module in Python.

YIQ

This color space is used for TV/video in the United States and other countries where NTSC is the video standard (Australia uses PAL). In this scheme, Y is the "luminance" (this corresponds roughly with intensity), and I and Q carry the color information. The conversion between RGB is straightforward:

$$
\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ 0.596 & -0.274 & -0.322 \\ 0.211 & -0.523 & 0.312 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}
$$

and

$$
\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1.000 & 0.956 & 0.621 \\ 1.000 & -0.272 & -0.647 \\ 1.000 & -1.106 & 1.703 \end{bmatrix} \begin{bmatrix} Y \\ I \\ Q \end{bmatrix}
$$

The two conversion matrices are of course inverses of each other. Note the difference between Y and V:

$$
\begin{aligned}
Y &= 0.299R + 0.587G + 0.114B \\
V &= \max\{R, G, B\}
\end{aligned}
$$

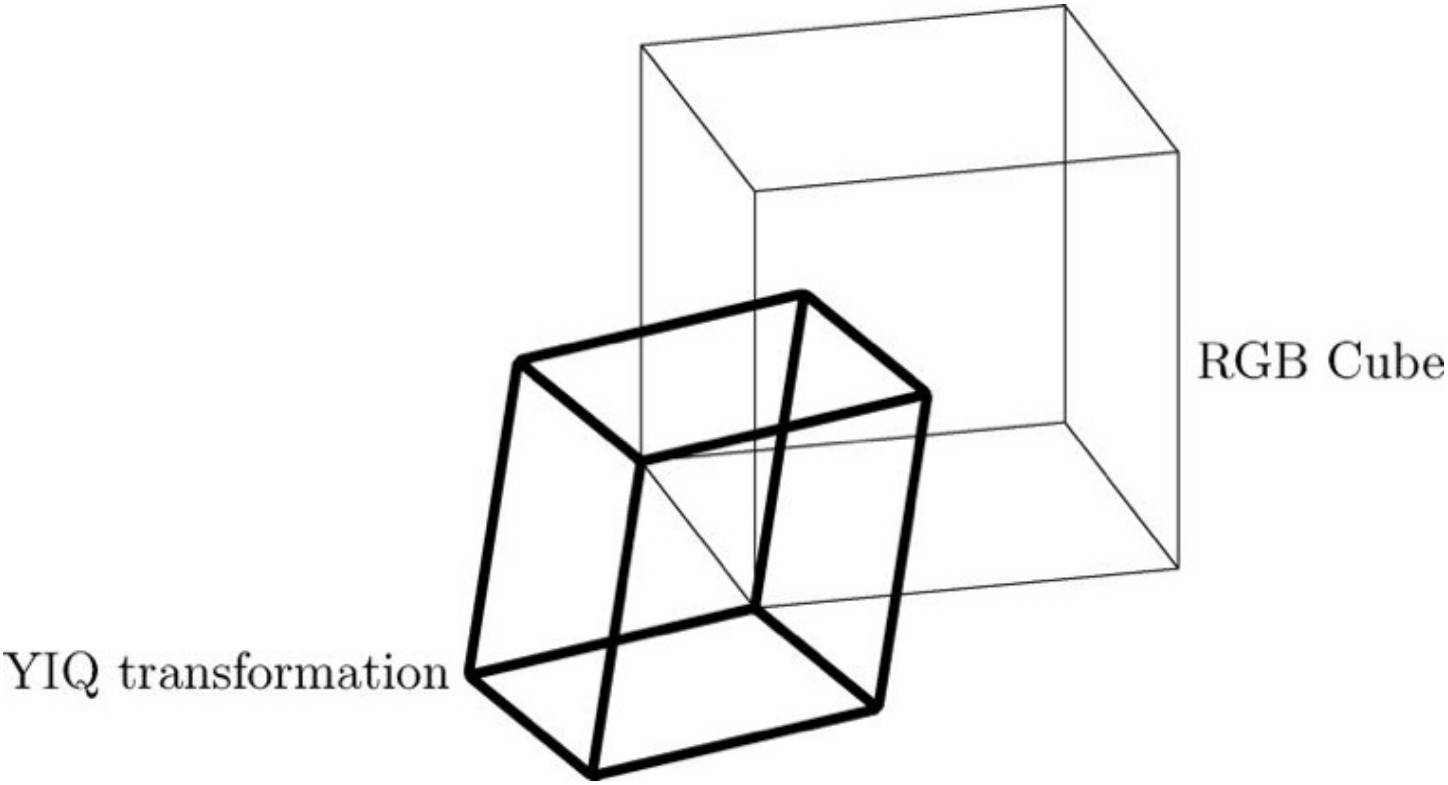
This reflects the fact that the human visual system assigns more intensity to the green component of an image than to the red and blue components. We note here that other transformations [13] $RGB \leftrightarrow HSV$ have

$$V = 0.333R + 0.333G + 0.333B$$

where the intensity is a simple average of the primary values. Note also that the Y of YIQ is different to the Y of XYZ, with the similarity that both represent luminance.

Since YIQ is a linear transformation of RGB, we can picture YIQ to be a parallelepiped (a rectangular box that has been skewed in each direction) for which the Y axis lies along the central (0, 0, 0) to (1, 1, 1) line of RGB. Figure 13.6 shows this.

Figure 13.6: The RGB cube and its YIQ transformation



That the conversions are linear, and hence easy to do, makes this a good choice for color image processing. Conversion between RGB and YIQ is implemented with the MATLAB/Octave functions `rgb2ntsc` and `ntsc2rgb`. In Python, we can use the `rgb_to_yiq` and `yiq_to_rgb` functions in the standard `colorsys` module.

13.3 Manipulating Color Images

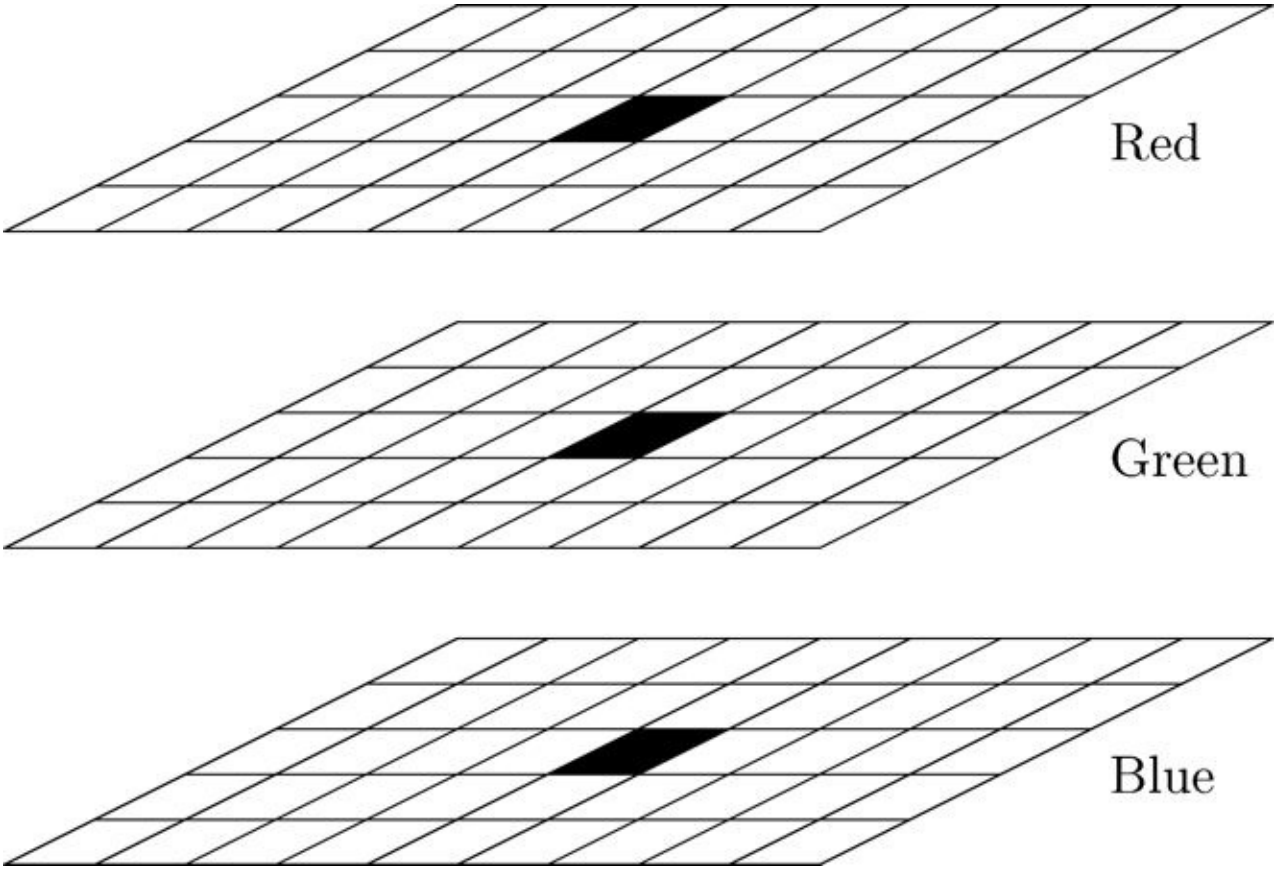
Since a color image requires three separate items of information for each pixel, a (true) color image of size $m \times n$ can be represented by an array of size $m \times n \times 3$: a three-dimensional array. We can think of such an array as a single entity consisting of three separate matrices aligned vertically. Figure 13.7 shows a diagram illustrating this idea. Suppose we read in an RGB image:

```
>> x = imread('twins');
>> size(x)

ans =

   256   256     3
```

MATLAB/Octave

or with

Figure 13.7: A three-dimensional array for an RGB image




```
In :  x = io.imread('twins.png')
In :  x.shape
Out:  (256, 256, 3)
```

Python

Each color component can be isolated by the colon operator:

MATLAB/Octave	Python	
`x(:,:,1)`	`x[:,:,0]`	The first, or red component
`x(:,:,2)`	`x[:,:,1]`	The second, or green component
`x(:,:,3)`	`x[:,:,2]`	The third, or blue component

These can all be viewed:

```
>> imshow(x)
>> for i = 1:3, figure(i+1),imshow(x(:,:,i)),end
```

MATLAB/Octave

or in Python with

```
In :  io.imshow(x)
In :  for i in range(3):
...:      plt.figure(i+2); io.imshow(x[:,:,i])
...:
```

Python

(Note that although arrays and lists in Python are indexed starting at 0, figures are automatically indexed starting at 1.)

These are all shown in Figure 13.8. Notice how the colors with particular hues show up with high intensities in their respective components. The orange clothing shows up as very light colored, as it is mostly red, in the red component. In the green component, that orange clothing appears dark in comparison with the yellow clothing. This is because yellow consists of more green than does orange. There are no particularly bright patches in the blue component, but the clothing now appears very dark, indicating the small amount of blue in each of the two colors.

We can convert to YIQ or HSV and view the components again:

Figure 13.8: SEE COLOR INSERT An RGB color image and its components

A color image | Red component | Green component | Blue component

```
>> xh = rgb2hsv(x);
>> for i = 1:3, figure(i+1),imshow(xh(:,:,i)),end
```

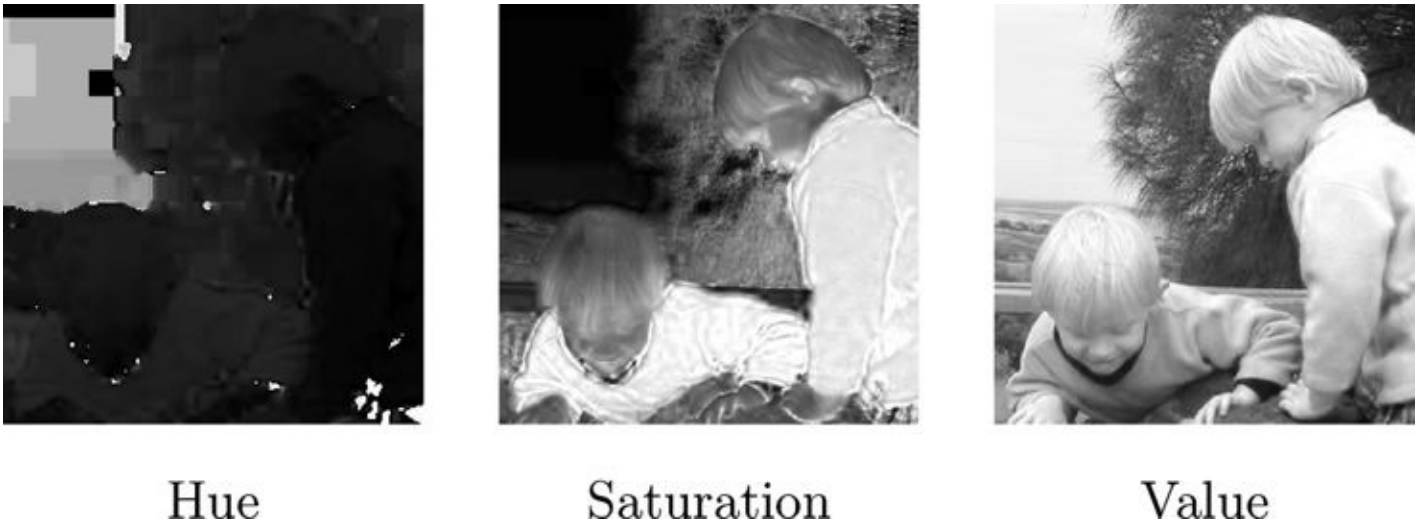
MATLAB/Octave

```
In :  xh = sk.color.rgb2hsv(x)
In :  for i in range(3):
...:      plt.figure(i); io.imshow(xh[:,:,i])
...:
```

Python

and these are shown in Figure 13.9. We can do precisely the same thing for the YIQ color

Figure 13.9: The HSV components



space:

```
>> xyiq = rgb2ntsc(x);
>> for i = 1:3, figure(i+1),imshow(xyiq(:,:,i)),end
```

MATLAB/Octave

In Python, the function to use is `rgb_to_yiq`, which is part of the `colorsys` module and which takes not just the image but the individual color components as inputs. To obtain the components quickly the `dsplit` command can be used to split the 256 × 256 × 3 RGB array into a list of three separate arrays of size 256 × 256 × 1. Preceding the `dsplit` function with an asterisk "unpacks" the elements of the list into individual elements, which can be used as inputs to `rgb_to_yiq`.

```
In :  xyiq = colorsys.rgb_to_yiq(*np.dsplit(x,3))
In :  for i in range(3):
...:      plt.figure(i); io.imshow(xyiq[i][:,:,0])
...:
```

Python

Note that the same result could be obtained simply by multiplying each column of the matrix `x` by the RGB to YIQ conversion matrix:

```
In :  rgb2yiq = np.array
    ([[.299,.587,0.114],[.596,-.275,-.321],[.212,-.528,.311]])
In :  xyiq = x.dot(rgb2yiq)
In :  for i in range(3):
...:      plt.figure(i); io.imshow(xyiq[:,:,i])
...:
```

Python

The Y, I, and Q images are shown in Figure 13.10. Notice that the Y component of YIQ gives a better grayscale version of the image than the value of HSV. The clothing, in particular, is quite washed out in

Figure 13.9 (value), but shows better contrast in Figure 13.10 (Y).

Figure 13.10: The YIQ components

We shall see below how to put three matrices, obtained by operations on the separate components, back into a single three dimensional array for display.

13.4 Pseudocoloring

This means assigning colors to a grayscale image in order to make certain aspects of the image more amenable for visual interpretation—for example, for medical images. There are different methods of pseudocoloring.

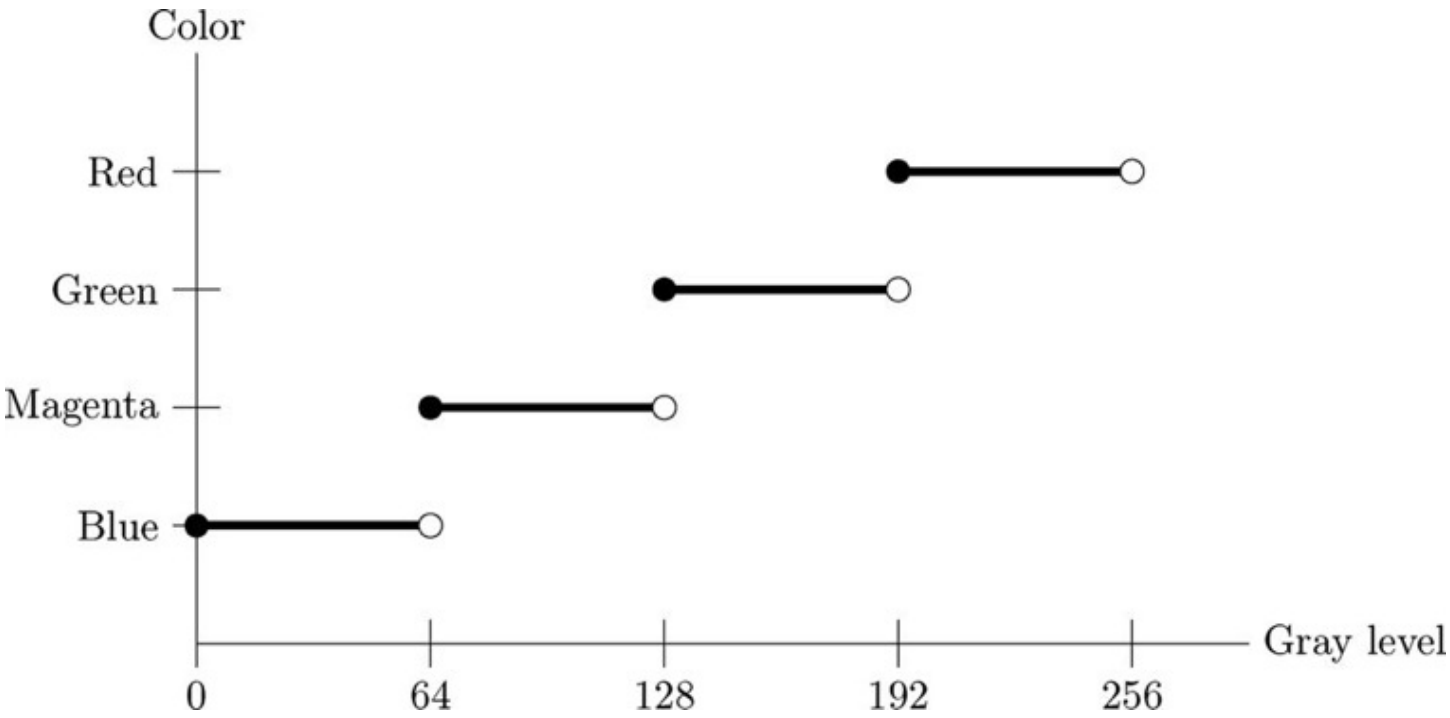
Intensity Slicing

In this method, the image is split into various gray level ranges, and a different color is assigned to each range. For example:

Gray level:	0–63	64–127	128–191	192–255
Color:	Blue	Magenta	Green	Red

This can be considered as a mapping, as shown in Figure 13.11.

Figure 13.11: Intensity slicing as a mapping



Gray-Color Transformations

We have three functions $f_R(x), f_G(x), f_B(x)$ which assign red, green, and blue values to each gray level x. These values (with appropriate scaling, if necessary) are then used for display. Using an appropriate set of functions can enhance a grayscale image with impressive results. An example is shown in Figure 13.12.

The gray level x in the diagram is mapped onto red, green and blue values of approximately 0.8, 0.7, and 0.3, respectively.

In MATLAB or Octave, a simple way to view an image with added color is to use `imshow` with an extra `colormap` parameter. For example, consider the image `blocks.png`. We can add a color map with the `colormap` function; there are several existing color maps to choose from. Figure 13.13 shows the children's blocks image (from Figure 1.4) after color transformations. Color image (a) can be created with:

```
>> b = imread('blocks.png');
>> imshow(b,colormap(jet(256))
```

MATLAB/Octave

or with:

```
In :  b = io.imread('blocks.png')
In :  io.imshow(b,cmap=plt.cm.jet)
```

Python

However, a bad choice of color map can ruin an image. Image (b) in Figure 13.13 is an example of this, where we apply the `vga` color map. This map in fact is not included in Octave, so here it is:

```
>> x = [1 1 1;1 0 0;1 1 0;0 1 0;0 1 1;0 0 1;1 0 1;0 0 0]
>> vga = [4*x(1,:);3 3 3;4*x(2:8,:);2*x(1:7,:)]/4
```

Octave

Figure 13.12: Mapping grays to colors

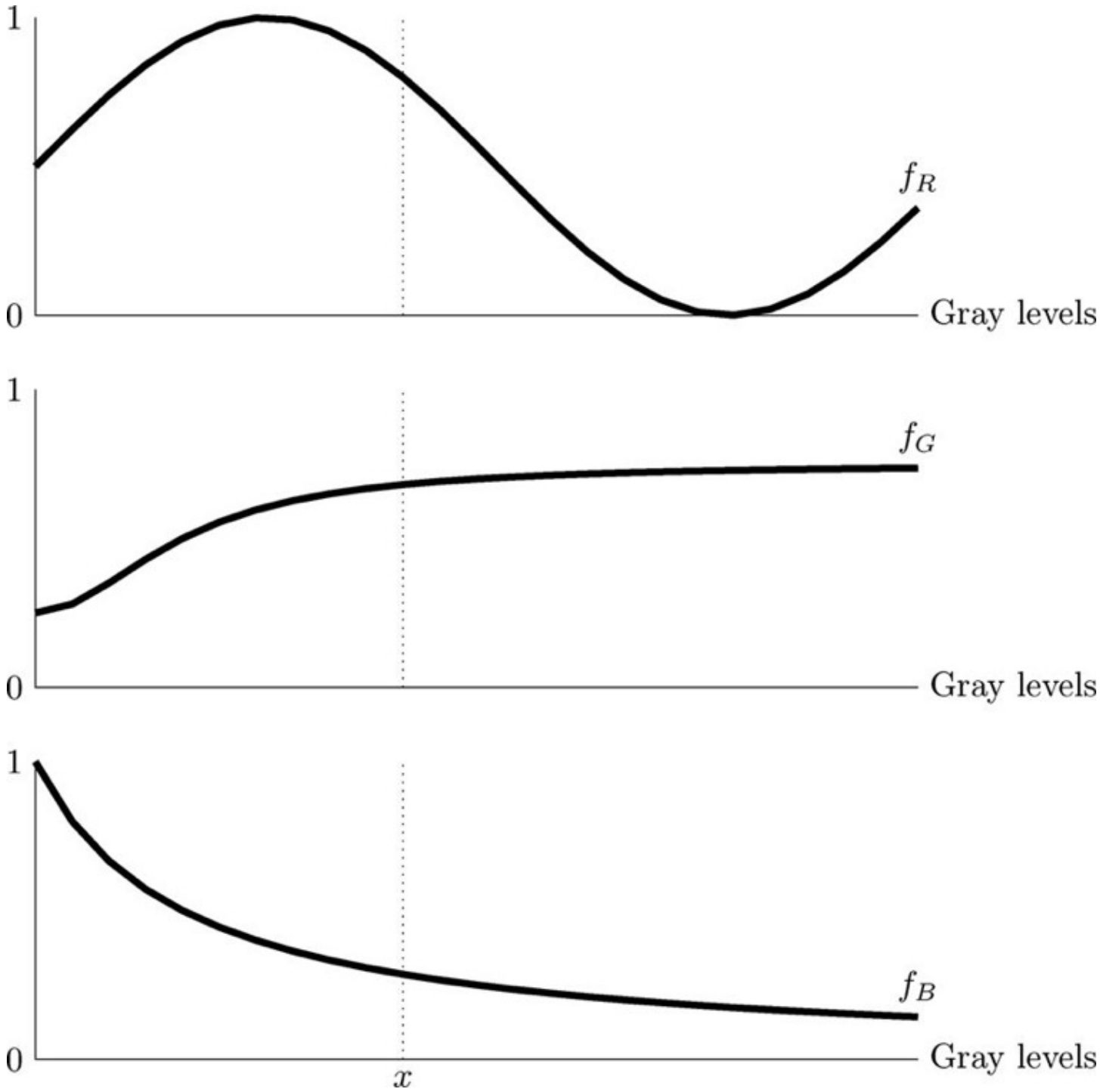
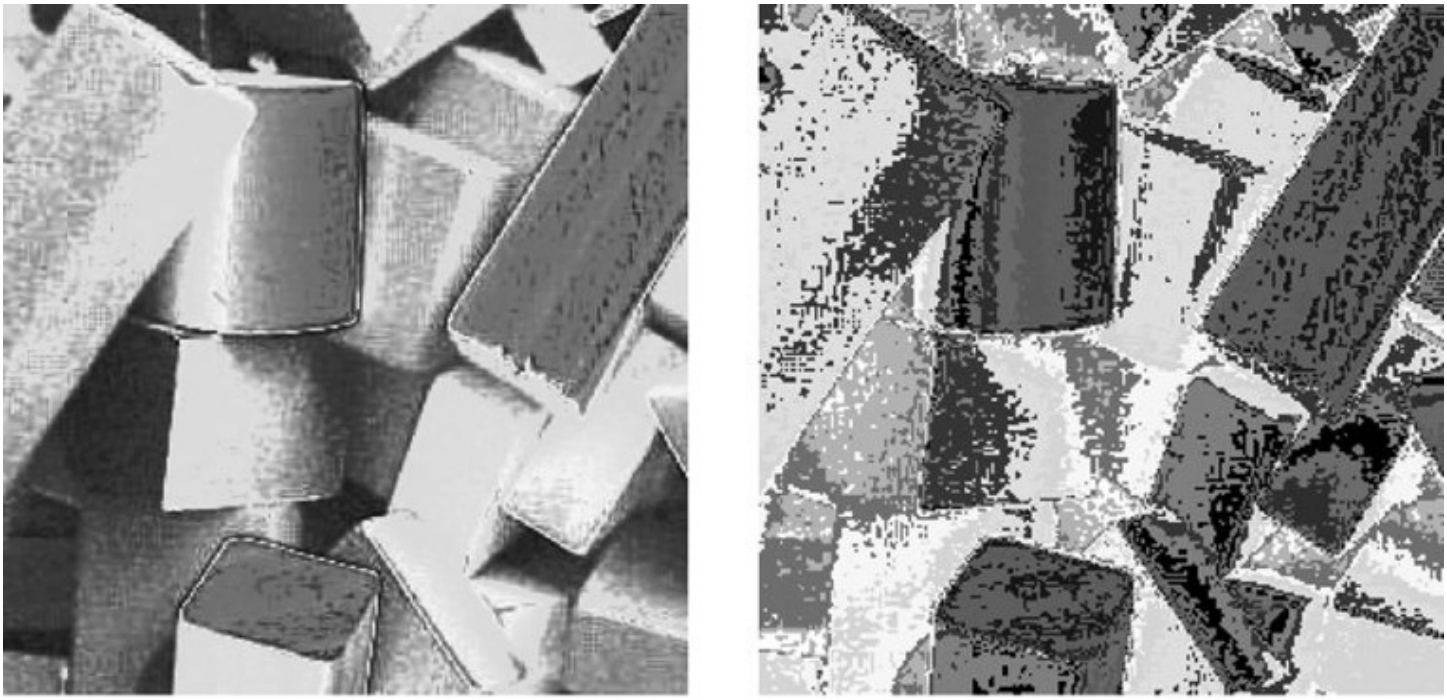


Figure 13.13: SEE COLOR INSERT Applying a color map to a grayscale image



Since this only has 16 rows, we need to reduce the number of grayscales in the image to 16. This can be done with the `grayslice` function:

```
>> b16 = grayslice(im2double(b),16);
>> figure,imshow(b16+1,colormap(vga))
```

MATLAB/Octave

In Python, the same effect can be obtained by first creating the array of VGA colors, and then turning that array into a colormap using the `vstack` function, which stacks arrays vertically:

```
In :  x = np.array([[1, 1, 1],[1, 0, 0],[1, 1, 0],[0, 1, 0],[0, 1, 1],[0,
    0, 1],[1,
0, 1],[0, 0, 0]])
In :  vga = np.vstack((4*x[0,:],np.array([[3, 3, 3]]),4*x[1:,:],2*x[:7,:]))
    /4.0
In :  b16 = b/16
In :  myvga = mpl.colors.ListedColormap(vga)
In :  io.imshow(b16,cmap=myvga)
```

Python

The result, although undeniably colorful, is not really an improvement on the original image. Each of MATLAB, Octave, and Python provide slightly different methods of listing the colormaps.

```
>> help graph3d
  Color maps.
    parula     - Blue-green-orange-yellow color map
    hsv        - Hue-saturation-value color map.
    hot        - Black-red-yellow-white color map.
    gray       - Linear gray-scale color map.
    bone       - Gray-scale with tinge of blue color map.
    copper     - Linear copper-tone color map.
    pink       - Pastel shades of pink color map.
    white      - All white color map.
    flag       - Alternating red, white, blue, and black color map.
    lines      - Color map with the line colors.
    colorcube  - Enhanced color-cube color map.
    vga        - Windows colormap for 16 colors.
    jet        - Variant of HSV.
    prism      - Prism color map.
    cool       - Shades of cyan and magenta color map.
    autumn     - Shades of red and yellow color map.
    spring     - Shades of magenta and yellow color map.
    winter     - Shades of blue and green color map.
    summer     - Shades of green and yellow color map.
```

MATLAB

```
>> colormap('list')
ns =
{
  [1,1] = autumn
  [1,2] = bone
  [1,3] = cool
  [1,4] = copper
  [1,5] = flag
  [1,6] = gmap40
  [1,7] = gray
  [1,8] = hot
  [1,9] = hsv
  [1,10] = jet
  [1,11] = lines
  [1,12] = ocean
  [1,13] = pink
  [1,14] = prism
  [1,15] = rainbow
  [1,16] = spring
  [1,17] = summer
  [1,18] = white
  [1,19] = winter
}
```

Octave

In Python, the colormaps are given as dictionaries stored as `datad` in the `cm` module of `matplotlib.pyplot`:

```
In :  import matplotlib.pyplot as plt
In :  plt.cm.datad.viewkeys()
Out:  dict_keys(['Spectral', 'summer', 'coolwarm', 'pink_r', 'Set1', 'Set2'
    , 'Set3',
'brg_r', 'Dark2', 'hot', 'PuOr_r', 'afmhot_r', 'terrain_r', 'PuBuGn_r', '
    RdPu',
'gist_ncar_r', 'gist_yarg_r', 'Dark2_r', 'YlGnBu', 'RdYlBu', 'hot_r',
'gist_rainbow_r', 'gist_stern', 'gnuplot_r', 'cool_r', 'cool', 'gray', '
    copper_r',
'Greens_r', 'GnBu', 'gist_ncar', 'spring_r', 'gist_rainbow', 'RdYlBu_r',
'gist_heat_r', 'OrRd_r', 'CMRmap', 'bone', 'gist_stern_r', 'RdYlGn', '
    Pastel2_r',
'spring', 'terrain', 'YlOrRd_r', 'Set2_r', 'winter_r', 'PuBu', 'RdGy_r', '
    spectral',
'flag_r', 'jet_r', 'RdPu_r', 'Purples_r', 'gist_yarg', 'BuGn', 'Paired_r',
    'hsv_r',
'bwr', 'cubehelix', 'YlOrRd', 'Greens', 'PRGn', 'gist_heat', 'spectral_r',
    'Paired',
'hsv', 'Oranges_r', 'prism_r', 'Pastel2', 'Pastel1_r', 'Pastel1', 'gray_r',
     'PuRd_r',
'Spectral_r', 'gnuplot2_r', 'BuPu', 'YlGnBu_r', 'copper', 'gist_earth_r', '
    Set3_r',
'OrRd', 'PuBu_r', 'ocean_r', 'brg', 'gnuplot2', 'jet', 'bone_r', '
    gist_earth',
'Oranges', 'RdYlGn_r', 'PiYG', 'CMRmap_r', 'YlGn', 'binary_r', 'gist_gray_r
    ',
```

Python

```
'Accent', 'BuPu_r', 'gist_gray', 'flag', 'seismic_r', 'RdBu_r', 'BrBG', '
    Reds', 'BuGn_r', 'summer_r', 'GnBu_r', 'BrBG_r', 'Reds_r', 'RdGy', '
    PuRd', 'Accent_r', 'Blues', 'Grays', 'autumn', 'cubehelix_r', '
    nipy_spectral_r', 'PRGn_r', 'Grays_r', 'pink', 'binary', 'winter', '
    gnuplot', 'RdBu', 'prism', 'YlOrBr', 'coolwarm_r', 'rainbow_r', '
    rainbow', 'PiYG_r', 'YlGn_r', 'Blues_r', 'YlOrBr_r', 'seismic', '
    Purples', 'bwr_r', 'autumn_r', 'ocean', 'Set1_r', 'PuOr', 'PuBuGn', '
    nipy_spectral', 'afmhot'])
```

Python

There are help files for each of these color maps, so that in both MATLAB and Octave the command

```
>> help jet
```

MATLAB/Octave

will provide some information on the `jet` color map. Python doesn't give any information as such, but the colormaps can be viewed with the `viewitems` method:

```
In :  plt.cm.datad['jet'].viewitems()
Out:  dict_items([('blue', ((0.0, 0.5, 0.5), (0.11, 1, 1), (0.34, 1, 1),
    (0.65, 0, 0), (1, 0, 0))), ('green', ((0.0, 0, 0), (0.125, 0, 0),
    (0.375, 1, 1), (0.64, 1, 1), (0.91, 0, 0), (1, 0, 0))), ('red', ((0.0,
     0, 0), (0.35, 0, 0), (0.66, 1, 1), (0.89, 1, 1), (1, 0.5, 0.5)))])
```

Python

We can easily create our own color map: it must by a matrix with 3 columns, and each row consists of RGB values between 0.0 and 1.0. Suppose we wish to create a blue, magenta green, red color map as shown in Figure 13.11. Using the RGB values:

Color	Red	Green	Blue
Blue	0	0	1
Magenta	1	0	1
Green	0	1	0
Red	1	0	0

we can create our color map with:

```
>> mycolormap = [0 0 1;1 0 1;0 1 0;1 0 0];
```

MATLAB/Octave

Before we apply it to the blocks image, we need to scale the image down so that there are only the four grayscales 0, 1, 2, and 3:

```
>> b4 = grayslice(im2double(b),4);
>> imshow(b4+1,mycolormap)
```

MATLAB/Octave

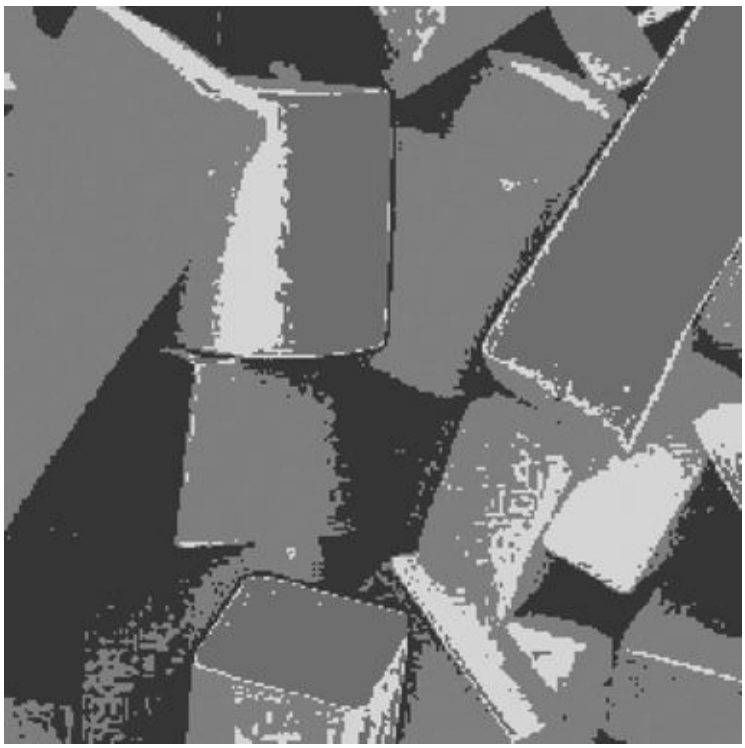
In Python, the method is slightly more complex:

```
In :  b = io.imread('blocks.png')
In :  b4 = b/64
In :  mycolormap = mpl.colors.ListedColormap([[0, 0, 1],[1, 0, 1],[0, 1,
    0],[1, 0, 0]])
In :  io.imshow(b4,cmap=mycolormap)
```

Python

and the result is shown in Figure 13.14.

Figure 13.14: SEE COLOR INSERT An image colored with a "handmade" color map



13.5 Processing of Color Images

There are two methods we can use:

1. We can process each R, G, B matrix separately.
2. We can transform the color space to one in which the intensity is separated from the color, and process the intensity component only.

Schemas for these are given in Figure 13.15.

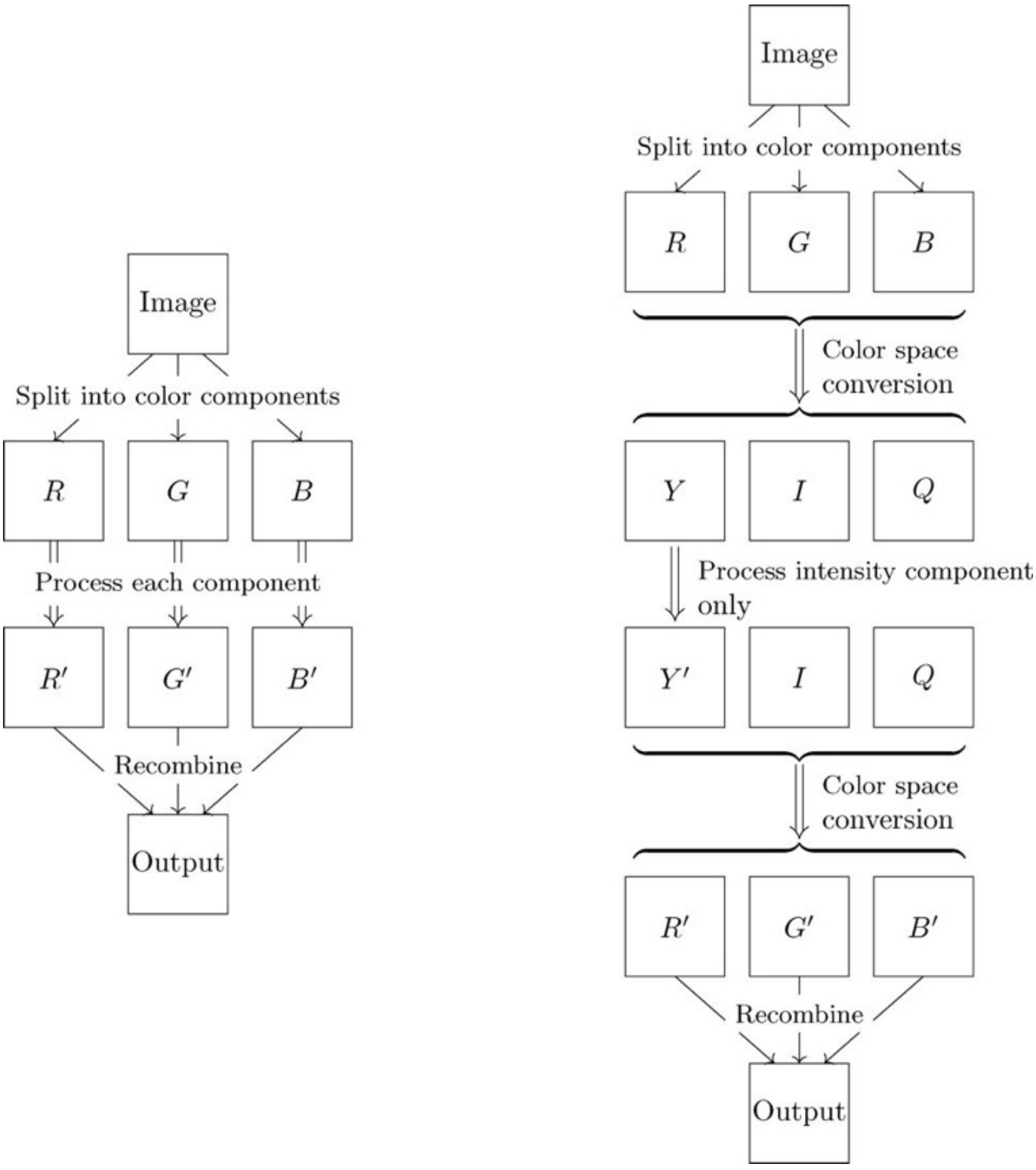
We shall consider a number of different image processing tasks, and apply either of the above schema to color images.

Contrast Enhancement

We know that histogram equalization, in general, provides a simple method of contrast enhancement. So one way of enhancing the contrast in a color image would be to enhance each of the color layers separately. For example, take the image `color_sunset.png`, which is the original color image that we

used before as a grayscale image in Chapter 4. Each layer can be processed separately, and the final result viewed. In MATLAB or Octave:

Figure 13.15: Color processing




```
>> s = imread('color_sunset.png');
>> s2 = zeros(size(s));
>> for i = 1:3, s2(:,:,i) = histeq(s(:,:,i)); end
>> imshow(s2)
```

MATLAB/Octave

or in Python:

```
In :  s = io.imread('color_sunset.png').astype('float64')
In :  s2 = np.zeros_like(s)
In :  for i  in range(3):
...:      s2[:,:,i] = sk.exposure.equalize_hist(s[:,:,i])
...:
In :  io.imshow(s2)
```

Python

Both images are shown in Figure 13.16.

Figure 13.16: SEE COLOR INSERT An attempt at color contrast enhancement

(a) The original image

(b) Attempt at equalization

Note that although the result is certainly better contrasted, there are some curious new colors introduced: the orange sheen on the water has been replaced with a brownish-white and in general both water and sky are "washed out," and there is a light purple tint in the clouds.

A better result here may be obtained by decoupling the intensity information from the intensity component, for example, by converting from RGB to YIQ, and then processing the Y component only:

```
>> syiq = rgb2ntsc(s);
>> syiq(:,:,1) = histeq(syiq(:,:,1));
>> s2 = ntsc2rgb(syiq);
```

MATLAB/Octave

For simplicity, instead of using *Y* in TIQ for the intensity information, in Python choose instead the V from HSV, and import the `color` module first:

```
In :  import skimage.color as co
In :  sh = co.rgb2hsv(s)
In :  sh[:,:,2] = sk.exposure.equalize_hist(sh[:,:,2])
In :  s2 = co.hsv2rgb(sh)
```

Python

The result is shown in Figure 13.17

Figure 13.17: SEE COLOR INSERT A better attempt at color contrast enhancement

Note that the colors here are much more “true to life” than in the original attempt. One more example is of an emu; Figure 13.18 shows the original, and the enhancement with all RGB layers, or with an intensity layer only.

Figure 13.18: SEE COLOR INSERT Another example of color contrast enhancement

(a) The original image (b) Processing each RGB (c) Processing Y only

In this case, there is only minimal visible difference, mainly because the initial image is quite well contrasted already.

Spatial Filtering

It very much depends on the filter as to which schema we use. In general, if the filter will not dramatically change the values, it is probably safe to use either schema. For example, consider an image of a koala, `koala_color.png`, loaded and stored as a three-dimensional array `k`. Now try unsharp masking, first with each RGB component, and then with an intensity component only. In MATLAB and Octave, just use the unsharp filter given by `fspecial`.

```
>> u = fspecial('unsharp');
>> k2 = zeros(size(k));
>> for i = 1:3, k2(:,:,i) = imfilter(k(:,:,i),u); end
>> imshow(k2/255)
>> kyiq = rgb2ntsc(k);
>> kyiq(:,:,1) = imfilter(kyiq(:,:,1),u);
>> k3 = ntsc2rgb(kyiq);
>> figure, imshow(k3)
```

MATLAB/Octave

In Python the same filter can be used, but it needs to be entered first.

```
In :  u = np.array([[-1,-4,-1],[-4,26,-4],[-1,-4,-1]])/6.0
In :  k2 = np.zeros_like(k)
In :  for i in range(3):
...:      k2[:,:,i] = ndi.convolve(k[:,:,i],u)
...:
In :  kh = co.rgb2hsv(k)
In :  kh[:,:,2] = ndi.convolve(kh[:,:,2],u)
In :  k3 = co.hsv2rgb(kh)
```

Python

and the results are shown in Figure 13.19.

Figure 13.19: SEE COLOR INSERT Unsharp masking of a color image

(a) The original image (b) Processing each RGB (c) Processing Y only

As an example of a low pass filter consider a Kuwahara filter, first on each RGB component and then on the intensity component only. Note that the Kuwahara implementation is entirely unoptimized, so this is a very slow process!

```
>> c = imread('cat.png');
>> ck = zeros(size(c));
>> for i = 1:3, ck(:,:,i) = kuwahara(c(:,:,i)); end
>> imshow(ck)
>> cyiq = rgb2ntsc(c);
>> cyiq(:,:,1) = kuwahara(cyiq(:,:,1));
>> c3 = ntsc2rgb(cyiq);
>> figure, imshow(c3)
```

MATLAB/Octave

The Python approach is very similar, as it was for unsharp masking, but for intensity again use the V from HSV:

```
In :  ck = np.zeros_like(c)
In :  for i in range(3):
...:      ck[:,:,i] = kuwahara(c[:,:,i])
...:
In :  ch = co.rgb2hsv(c)
In :  ch[:,:,2] = kuwahara(ch[:,:,2])
In :  c3 = co.hsv2rgb(ch)
```

Python

The results are shown in Figure 13.20, and as before there is not a huge observable difference between the outputs.

Figure 13.20: SEE COLOR INSERT Kuwahara filtering of a color image

(a) The original image (b) Processing each RGB (c) Processing Y only

Noise Reduction

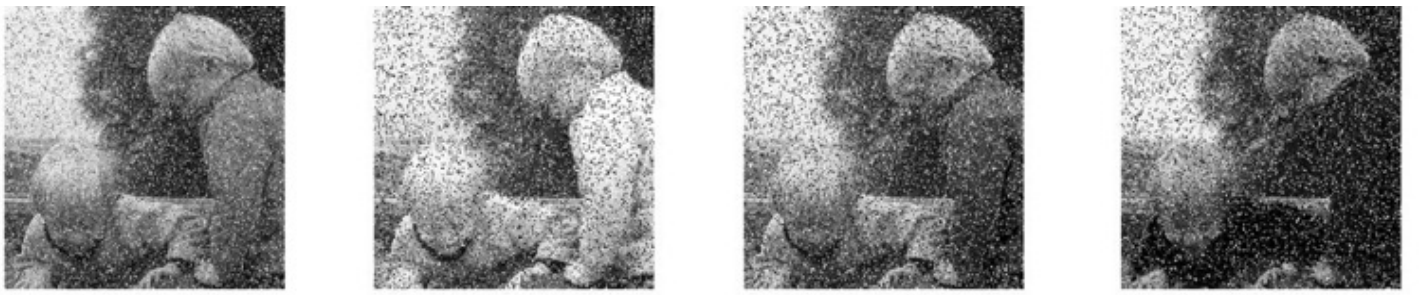
Consider the twins image, with salt and pepper noise added:

```
>> tn = imnoise(t,'salt_&_pepper',0.2);
>> imshow(tn)
>> figure,imshow(tn(:,:,1))
>> figure,imshow(tn(:,:,2))
>> figure,imshow(tn(:,:,3))
```

MATLAB/Octave

These are all shown in Figure 13.21. It would appear that median filtering should be applied to each of the RGB components. This is easily done, using exactly the same commands as for Kuwahara filtering and histogram equalization, except for using `medfilt2` instead of `kuwahara` or `histeq` (or `equalize_hist`).

Figure 13.21: SEE COLOR INSERT Noise on a color image



Salt and pepper noise | The red component | The green component | The blue component

The result is shown in Figure 13.22(a). In this instance we cannot apply the median filter to the intensity component only, because the conversion from RGB to YIQ spreads the noise across all the YIQ components. If we remove the noise from *Y* only, then the noise has been only slightly diminished as shown in Figure 13.22(b). If the noise applies to only one of the RGB components, then it would be appropriate to apply a denoising technique to this component only.

Figure 13.22: SEE COLOR INSERT Attempts at denoising a color image

(a) Denoising each RGB component (b) Denoising intensity only

Also note that the method of noise removal must depend on the generation of noise. In the above example, we tacitly assumed that the noise was generated after the image had been acquired and stored as RGB components. But as noise can arise anywhere in the image acquisition process, it is quite reasonable to assume that noise might affect only the brightness of the image. In such a case, denoising an intensity component only will produce the best results.

Edge Detection

An edge image will be a binary image containing the edges of the input. We can go about obtaining an edge image in two ways:

1. We can take the intensity component only, and find its edges.
2. We can find the edges of each of the RGB components, and join the results.

To implement the first method in MATLAB or Octave, we start with the `rgb2gray` function, applied to an image of a Venetian canal:

```
>> v = imread('venice.png');
>> vg = rgb2gray(v);
>> ve = edge(vg);
>> imshow(ve)
```

MATLAB/Octave

In Python the `sobel` function can be used with a threshold value:

```
In :  v = io.imread('venice.png')
In :  vg = co.rgb2gray(v)
In :  ve1 = fl.sobel(vg)>0.11
```

Python

Recall that the `edge` function with no parameters implements Sobel detection. The result is shown in Figure 13.23(a). For the second method, we can join the results with the logical "or":

```
>> v2 = zeros(size(v));
>> for i = 1:3, v2(:,:,i) = edge(v(:,:,i)); end
>> ve2 = ve2 = v2(:,:,1) | v2(:,:,2) | v2(:,:,3);
>> figure,imshow(ve2)
```

MATLAB/Octave

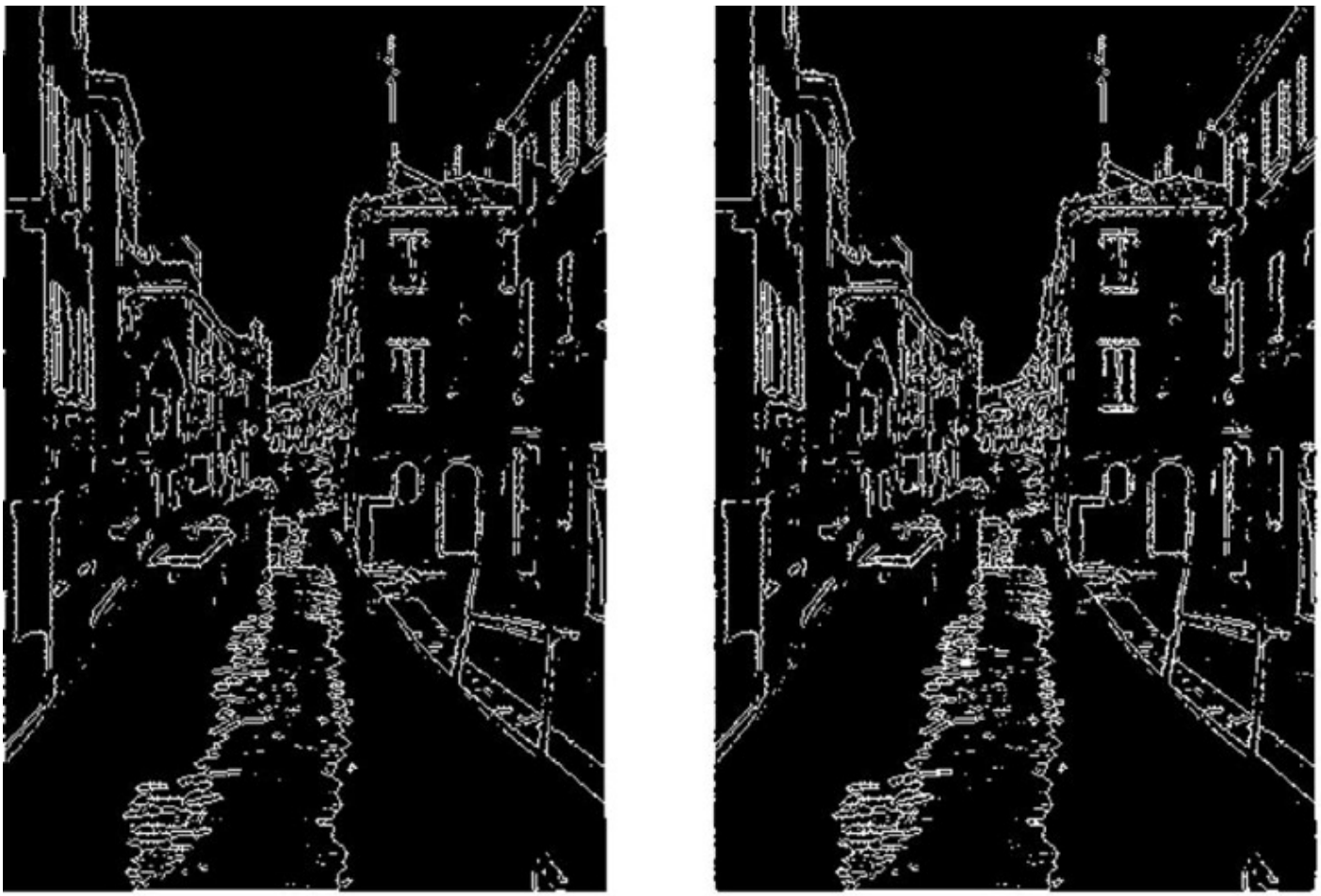
The method is similar in Python, except that since `v2` is a numeric array, the results of the Sobel filtering will be cast to numeric values:

```
In :  v2 = np.zeros_like(v)
In :  for i in range(3):
...:      v2[:,:,i] = fl.sobel(v[:,:,i])>0.11
...:
In :  ve2 = (v2[:,:,0] + v2[:,:,1] + v2[:,:,2])>0
```

Python

and this is shown in Figure 13.23(b). The edge image `ve2` is a much more complete edge image. Frames of doors and windows are more complete, as are roof edges. However, the success of these methods will also depend on the parameters of the edge function chosen; for example, the threshold value used. In the examples shown, the `edge` function has been used with its default threshold.

Figure 13.23: The edges of a color image



`ve1`: Edges after `rgb2gray` `ve2`: Edges of each RGB component

The Retinex Algorithm

In the 1970s Edwin Land (of Polaroid camera fame) proposed a "retinex" theory of human vision [27]; the word being a mix of "retina" and "cortex." Retinex algorithms, of which there are many, have much in common with homomorphic filtering which was discussed in Section 7.9. Retinex algorithms are designed to improve visual contrast, such as in dark shadowed areas of an image, while maintaining color.

One particular algorithm, known as "center/surround retinex" [25], works by subtracting from the logarithm of the image the logarithm of the image filtered with a Gaussian filter:

$$R(x,y) = \log I(x,y) - \log\left(I(x,y) * F(x,y)\right).$$

The filtering is usually done using the Fourier transform, as one version uses a filter the same size as the image itself.

Figure 13.24(a) shows a street scene taken in low light (through a car window). It can be processed in MATLAB or Octave as:

```
>> s = im2double(rgb2gray(imread('street.png')));
>> s1 = mat2gray(s)+0.01;
>> f = fspecial('gaussian',size(s),40);
>> sf =  abs(ifft2(fft2(s1).*fft2(f)));
>> sf(find(sf==0))=0.01;
>> s2 = log(s1) - log(sf);
>> imshow(mat2gray(sf))
```

MATLAB/Octave

or in Python as

```
In :  s = ut.img_as_float(co.rgb2gray(io.imread('street.png')))
In :  s1 = ex.rescale_intensity(s,out_range=(0,1))+0.01
In :  r,c = s.shape
In :  i,j = np.mgrid[-(r//2):(r+1)//2,-(c//2):(c+1)//2]
In :  f = np.exp(-(i**2+j**2)/40**2)
In :  sf = abs(ifft2(fft2(s1)*fft2(f)))
In :  sf[np.where(sf==0)]=0.01
In :  s2 = np.log(s1) - np.log(sf)
In :  io.imshow(s2)
```

Python

and the result is shown in Figure 13.24(b).

For a color image, there is the option of either processing each RGB component, or the intensity component only from either HSV or YIQ. Figure 13.25 shows a picture with a lot of dark shadows: on the far side of the ornamental lake it is hard to see any details. Figure 13.26 shows two outcomes of the center/surround retinex algorithm. This retinex algorithm is known as a “single-scale retinex” as it uses a single Gaussian filter; more sophisticated algorithms use multiple scales—different Gaussian filters—and can preserve color intensity and saturation.

Programs to implement this single-scale retinex are given below.

Figure 13.24: An application of center/surround retinex

(a) A "dull" image (b) Result of retinex processing

Figure 13.25: SEE COLOR INSERT The botanic gardens, Melbourne, Australia

Figure 13.26: SEE COLOR INSERT Center/surround retinex on a color image

(a) Applying retinex to each of RGB (b) Applying retinex to Y only

13.6 Programs

Here are programs for computing and displaying the RGB gamut, first in MATLAB or Octave:

```
function plotgamut()
  xyz2rgb = [3.063 -1.393 -0.476; -0.969 1.876 0.042;0.068 -0.229 1.069];
  ciexyz = csvread('ciexyz31.csv');
  ciexyz = [ciexyz;ciexyz(1,:)];
  W = ciexyz(:,1);
  X = ciexyz(:,2);
  Y = ciexyz(:,3);
  Z = ciexyz(:,4);
  S = X+Y+Z;

  [xx,yy] = meshgrid(1:512);
  z0 = zeros([size(xx),3]);
  z0(:,:,1)=xx/512;
  z0(:,:,2) = yy/512;
  z0(:,:,3) = 1-xx/512-yy/512;
  z0r = reshape(z0,prod(size(xx)),3)';
  rgb0 = xyz2rgb*z0r;
  mn = min(rgb0)<0;
  rgb1 = max(rgb0,[mn;mn;mn]);
  % rgb1(find(rgb1>1))=1;
  % mm = min(rgb0)>=0 & max(rgb0)<=1;
  % rgb1 = rgb0.*[mm;mm;mm];
  % rgb1(find(rgb1==0))=1;
  rgbs = reshape(rgb1',[size(xx),3]);
  figure,imshow(flipdim(rgbs,1))
  %figure,imshow(rgbs)
  hold on
  plot(floor(X./S*512),512-floor(Y./S*512),'-k','linewidth',3),axis on
  %plot(floor(X./S*512),floor(Y./S*512),'-k','linewidth',3),axis on
end
```

MATLAB/Octave

and second in Python:

```
def plotgamut():
  xyz2rgb = np.array([[3.063, -1.393, -0.476],[-0.969, 1.876,
      0.042],[0.068, -0.229, 1.069]])
  ciexyz = np.loadtxt('ciexyz31_1.csv',delimiter=',')
  ciexyz1 = np.vstack((ciexyz,ciexyz[0,:,np.newaxis].T))
  W = ciexyz1[:,0]
  X = ciexyz1[:,1]
  Y = ciexyz1[:,2]
  Z = ciexyz1[:,3]
  S = X+Y+Z

  xx,yy = np.mgrid[0:512,0:512].astype('float64')
  z0 = np.zeros((512,512,3))
```

Python

```
    z0[:,:,0] = xx/512;
    z0[:,:,1] = yy/512;
    z0[:,:,2] = 1.0-xx/512-yy/512;
    z0r = np.reshape(z0,(xx.size,3),order='F').T
    rgb0 = xyz2rgb.dot(z0r)
    mn = (rgb0.min(axis=0)<0)*1.0
    rgb1 = np.maximum(rgb0,np.tile(mn,(3,1)))
    # rgb1(find(rgb1>1))=1;
    # mm = min(rgb0)>=0 & max(rgb0)<=1;
    # rgb1 = rgb0.*[mm;mm;mm];
    # rgb1(find(rgb1==0))=1;
    rgbs = np.reshape(rgb1.T,(xx.shape+(3,)),order='F');
    rgbs[np.where(rgbs>1)]=1.0
    #
    #figure,imshow(rgbs)
    plt.plot(np.ceil(X/S*512),512-np.ceil(Y/S*512))
    plt.imshow(np.flipud(rgbs),aspect='equal',origin='upper')
    plt.show()
    #plot(floor(X./S*512),floor(Y./S*512),"-k","linewidth",3),axis on
```

Python

The retinex algorithm in MATLAB or Octave:

```
% Single-scale retinex, adapted from
%   http://au.mathworks.com/MATLABcentral/fileexchange/26523
%
% Usage: ssretinex(image,hsize)
%
% where hsize is the spread of the Gaussian filter to be used
%
function out = ssretinex(X,hsize)
if length(size(X))==2
    imsize = size(X);
else
    imsize = size(X(:,:,1));
end
filt = fspecial('gaussian',imsize,hsize/sqrt(2));
if length(size(X))==2
    X1 = mat2gray(im2double(X))+0.05;
    Z = abs(ifft2(fft2(X1).*fft2(filt)));
    Z(find(Z==0))=0.05;
    L = log(Z);
    R = log(X1)-L;
    out = mat2gray(R);
else
    out = zeros(size(X));
    for i = 1:3
        X1 = mat2gray(im2double(X(:,:,i)))+0.01;
        Z = abs(ifft2(fft2(X1).*fft2(filt)));
        Z(find(Z==0))=0.01;
        L = log(Z);
        out(:,:,i) = mat2gray(log(X1)-L);
    end
end
```

MATLAB/Octave

```
end
```

MATLAB/Octave

and here is retinex in Python:

```python
def ssretinex(im,hsize):
    from scipy.fftpack import fft2,ifft2
    from skimage.exposure import rescale_intensity
    r,c = im.shape[:2]
    i,j = np.mgrid[-(r//2):(r+1)//2,-(c//2):(c+1)//2]
    f = np.exp(-((i**2+j**2)/hsize**2))
    if len(im.shape)==2:
        im1 = rescale_intensity(im.astype(float),out_range=(0,1))+0.01
        imf = abs(ifft2(fft2(im1)*fft2(f)))
        imf[np.where(imf==0)] = 0.01
        return np.log(im1) - np.log(imf)
    else:
        out = np.zeros_like(im).astype(float)
        im2 = im.astype(float)
        for i in range(3):
            imr = rescale_intensity(im2[:,:,i],out_range=(0,1))+0.01
            imf = abs(ifft2(fft2(imr)*fft2(f)))
            imf[np.where(imf==0)] = 0.01
            out[:,:,i] = rescale_intensity(np.log(imr) - np.log(imf),
                out_range=(0,1))
        return out
```

Python

Exercises

1. By hand, determine the saturation and intensity components of the following image, where the RGB values are as given:

$(0,1,1)$	$(1,2,3)$	$(7,7,7)$	$(5,1,2)$	$(1,1,7)$
$(2,1,2)$	$(1,7,7)$	$(2,0,2)$	$(3,3,2)$	$(5,5,0)$
$(4,4,4)$	$(4,6,7)$	$(4,5,6)$	$(1,5,7)$	$(3,6,7)$
$(3,0,3)$	$(5,2,2)$	$(1,1,1,$	$(6,6,0)$	$(2,2,2)$
$(1,2,1)$	$(0,4,4)$	$(3,1,6)$	$(3,3,3)$	$(2,4,6)$

2. Suppose the intensity component of an HSV image was thresholded to just two values. How would this affect the appearance of the image?

- 3. By hand, perform the conversions between RGB and HSV or YIQ, for the values:

R	G	B	H	S	V
0.5	0.5	0			
0	0.7	0.7			
0.5	0	0.5			
			0.33	0.5	1
			0.67	0.7	0.7
			0	0.2	0.8

R	G	B	Y	I	Q
0.3	0.3	0.7			
0.7	0.9	0			
0.8	0.8	0.7			
			1	0.3	0.3
			0.5	0.5	0.5
			0	1	1

You may need to normalize the RGB values.

- 4. Check your answers to the conversions in Question 3 by using the relevant functions in MATLAB, Octave, or Python.
- 5. Threshold the intensity component of a color image of your choice, and see if the result agrees with your guess from Question 2.
- 6. The image `emu.png` is an indexed color image. Experiment with applying different color maps to the index matrix of this image. Which color map seems to give the best results? Which color map seems to give the worst results?
- 7. View the image `street.png`. Experiment with histogram equalization on:

    (a) The intensity component of HSV
    (b) The intensity component of YIQ

Which seems to produce the best result?

- 8. Create and view a random "patchwork quilt" with:

```
>> r = uint8(floor(256*rand(16,16,3)));
>> r = imresize(r,16);
>> imshow(r),pixval on
```

MATLAB/Octave

or

```
In :  r = np.random.randint(0,256,(16,16,3))
In :  r = tr.resize(r.astype('uint8'),(256,256),order=0)
In :  io.imshow(r)
```

Python

What RGB values produce (a) a light brown color? (b) a dark brown color? Convert these brown values to HSV, and plot the hues on a circle.

- 9. Using the image `tropical_flower.png`, see if you can obtain an edge image from the intensity component alone, that is, as close as possible to the image `fe2` in Figure 13.23. What parameters to the `edge` function did you use? How close to `fe2` could you get?
- 10. Add Gaussian noise to an RGB color image `x` with

```
>> xn = imnoise(x,'gaussian');
```
MATLAB/Octave

or

```
In :  xn = skimage.util.random_noise(x)
```
Python

View your image, and attempt to remove the noise with

(a) Average filtering on each RGB component
(b) Wiener filtering on each RGB component

- 11. Take any color image of your choice and add salt and pepper noise to the intensity component. This can be done with

```
>> ty = rgb2ntsc(tw);
>> tn = imnoise(ty(:,:,1).'salt & pepper');
>> ty(:,:,1) = tn;
```
MATLAB/Octave

or

```
In :  rgb2yiq = np.array
    ([[.299,.587,0.114],[.596,-.275,-.321],[.212,-.528,.311]])
In :  yiq2rgb = np.linalg.inv(rgb2yiq)
In :  ty = ut.img_as_float(tw).dot(rgb2yiq)
In :  tn = ut.random_noise(ty[:,:,0],'s&p')
In :  ty[:,:,0] = tn
In :  t2 = ty.dot(yiq2rgb)
```
Python

Now convert back to RGB for display.

(a) Compare the appearance of this noise with salt and pepper noise applied to each RGB component as shown in Figure 13.21. Is there any observable difference?
(b) Denoise the image by applying a median filter to the intensity component.
(c) Now apply the median filter to each of the RGB components.
(d) Which one gives the best results?
(e) Experiment with larger amounts of noise.
(f) Experiment with Gaussian noise.

14 Image Coding and Compression

14.1 Lossless and Lossy Compression